SwegHammer

Project Handbook & Living To-Do List

Eddie Handford & Jake

Last updated: May 13, 2026

Contents

1 How to use this document

This is the single living reference for SwegHammer. Open it when you sit down, close it when you walk away. It contains the project vision, the math behind each combat phase, a checklist of everything left to do, and ownership tags so we both know who is on what.

Conventions

- ❍ open task ★ in progress ✓ done
- [E] Eddie's slice [J] Jake's slice [?] unclaimed

When you pick a task up, swap its tag to your initial and flip the marker to ★. When you finish, flip to ✓ and commit the `.tex` change alongside the code change so the doc never drifts from reality.

Build

```
pdflatex PROJECT.tex && pdflatex PROJECT.tex
```

Two passes so the table of contents resolves. No external assets needed.

2 Project vision

SwegHammer is a data-driven Warhammer 40K simulator with two goals:

1. Derive fair, transparent points costs from Lanchester's Square Law plus empirical simulation.
2. Test minimal rule tweaks (unit-by-unit activation, command-point compensation for the smaller army) that fix structural feel-bad moments like first-player advantage.

The simulator is the experimental apparatus. It must be *fast* enough to run thousands of battles in a calibration sweep, and *detailed* enough to render a single battle as a watchable replay for sanity-checking the model.

3 Where the project stands today

3.1 What works

- ✓ Stochastic combat engine (`code/simulator.py`): roll-to-hit, AP-adjusted armour save, cover modifier, damage.
- ✓ Unit catalogue (`code/units.py`) with ∼17 units across Marines, Chaos, Orks and Tyranids.
- ✓ Unit-by-unit alternating activation (`Battle._run_round`).
- ✓ Command-point bonus for the smaller army from Round 2.
- ✓ Streamlit dashboard (`app.py`) with preset battles, attrition curves, survivor histograms, points-vs-winrate sweep.

3.2 What's missing in one sentence

The current "combat" is a single abstract attack action. There are no *phases* (movement / shooting / charge / melee / morale), no map, no positions, and no terrain beyond a binary cover flag. The rest of this document is the plan to fill that in.

4 Game phases

40K splits a turn into discrete phases. We will mirror that structure but keep each phase pluggable so we can disable expensive ones for fast calibration runs.

4.1 Command phase

At the start of a unit's activation: gain 1 CP, resolve start-of-turn abilities, apply the smaller-army CP compensation rule.

To do

❍ [E] Add a `CommandPhase` step run at activation start.

❍ [E] Move existing CP bonus logic out of `Battle._run_round` into the new phase so it lives in one place.

4.2 Movement phase

A unit moves up to its Move characteristic in inches. Optionally Advance ($+d6$, no shooting that turn) or Fall Back (no shooting, no charging).

Math. Position update for a unit at $\mathbf{p}_t$ choosing destination $\mathbf{p}_{t+1}$ subject to

$$\|\mathbf{p}_{t+1} - \mathbf{p}_t\| \;\le\; M_{\text{unit}} \cdot \sigma_{\text{terrain}},$$

where $\sigma_{\text{terrain}} \in (0, 1]$ is the terrain-slowdown factor on the line the unit traverses.

To do

❍ [E] Add `position: (float, float)` to `Unit`.

❍ [E] Add `move: float` (inches) to `UnitProfile`.

❍ [E] Implement `MovementPhase` with a tactical heuristic: close on the nearest threat, claim the nearest objective, kite if outranged.

❍ [?] Advance roll ($+d6$) and Fall Back logic.

4.3 Shooting phase

For every weapon in range and with line of sight: roll to hit, roll to wound, target rolls armour save, apply damage. **The current sim skips the wound roll**, which is the biggest single bit of math debt.

Math. Expected damage of one shot:

$$\mathbb{E}[D] \;=\; P_{\text{hit}} \cdot P_w(S, T) \cdot \big(1 - P_{\text{save}}(\text{sv}, \text{AP}, \text{cover})\big) \cdot d,$$

where the wound probability P_w follows the strength-vs-toughness table:

$$P_w(S,T) = \begin{cases} 5/6 & S \ge 2T \\ 4/6 & S > T \\ 3/6 & S = T \\ 2/6 & S < T \\ 1/6 & 2S \le T \end{cases}$$

and the save probability uses an effective save characteristic $\text{sv}^* = \text{sv} - \text{AP} - \text{cover}$ capped at 2:

$$P_{\text{save}}(\text{sv}^*) \;=\; \max\!\Big(0,\; \tfrac{7-\text{sv}^*}{6}\Big).$$

To do

✓ [E] Add `strength: int` to `UnitProfile`.

✓ [E] Add `toughness: int` to `UnitProfile`.

✓ [E] Implement the wound table above as `wound_probability(S, T)` and wire it into `Unit.attack()` between the hit and save rolls.

❍ [E] Regenerate `BASELINE.md` catalogue table to reflect the wound-roll-aware point costs (existing table is now stale; mirror match still 50/50, but cross-faction matchups need recalibrating).

❍ [E] Add `range_inches: int` to the weapon profile; gate shooting on range and LoS.

4.4 Charge phase

Declare a charge against an enemy within 12″. Roll $2d6$; if the total is at least the distance to the target, move into base contact.

Math.

$$P_{\text{charge}}(d) \;=\; \Pr[2d_6 \geq d] \;=\; \frac{\#\{(a,b) \in [1,6]^2 : a+b \geq d\}}{36}.$$

Useful values: $P(6'') = 26/36 \approx 72\%$, $P(9'') = 10/36 \approx 28\%$, $P(12'') = 1/36 \approx 3\%$.

To do

❍ [?] Implement `ChargePhase` with declaration, roll and move.

❍ [?] Track an "engaged in melee" flag that suppresses shooting next turn.

4.5 Fight (melee) phase

Units in base contact fight. Each model makes `attacks` attacks at `weapon_skill`, then wound, save, damage as per shooting.

To do

❍ [?] Add `attacks_melee: int` and `weapon_skill: float` to `UnitProfile`.

❍ [?] Implement `MeleePhase`; resolve in fight-priority order (chargers fight first).

4.6 Morale / battleshock phase

A unit that lost models tests Leadership; a failed test inflicts further casualties or status effects.

To do

❍ [?] Add `leadership: int` to `UnitProfile`.

❍ [?] Decide between classic morale ($d6$ + losses vs Ld) and 10th-edition battleshock; document the choice in `THEORY.md`.

5 Terrain and map

5.1 Coordinate system

Continuous 2D plane, units in inches. Standard boards: $44'' \times 60''$ (Combat Patrol), $44'' \times 90''$ (Strike Force). Models are circles with a defined base radius.

5.2 Terrain types

Type	Effect	Notes
Open	none	default
Light cover	+1 save vs ranged	matches today's "in cover" flag
Heavy cover	+1 save and −1 to hit	ruins, barricades
Obscuring	blocks LoS through it	forests, walls
Impassable	blocks movement	cliffs

5.3 Line of sight

Naive implementation: draw a segment from the attacker centre to the target centre, check intersection with any obscuring polygon. If that becomes a perf bottleneck, swap in a spatial hash over terrain polygons.

To do

- ❍ [?] `Terrain` dataclass: polygon + type.
- ❍ [?] `Map` dataclass: board size, terrain list, objective markers.
- ❍ [?] `has_line_of_sight(attacker, target, map)`.
- ❍ [?] `cover_state(unit, map)` returning the cover type the unit currently occupies.
- ❍ [?] Two or three stock maps shipped with the repo for testing.

6 Visualization: the dual-mode plan

The simulator needs to do two things that pull in opposite directions:

1. Run thousands of fast battles for calibration sweeps.
2. Render a single battle in detail for sanity checks and demos.

Design. Keep the simulation core (`simulator.py`, `army.py`, `units.py`) *completely headless and deterministic given a seed.* Build rendering on top as an event-subscriber layer:

- **Detailed mode** – a *Pygame* or *matplotlib-animation* renderer with units drawn as tokens on a 2D top-down map, with range circles, LoS lines, charge arcs. Used for: replaying a single battle, demo videos, debugging movement bugs.
- **Fast mode** – *no rendering at all*, just numerical state. Used for: Monte Carlo sweeps where we run 10^3–10^4 battles per matchup and only care about win rates and attrition curves.

Crucially, the two modes share *the same* `Battle` object; the mode flag just toggles which subscriber is attached. A bug in the visualisation therefore cannot affect calibration numbers.

To do

- ❍ [?] Refactor `Battle.run()` to emit events (`UnitMoved`, `UnitShot`, `UnitKilled`, ...) via a subscriber list.
- ❍ [?] `NullRenderer` – no-op, the fast-mode default.
- ❍ [?] `PygameRenderer` – consumes the event stream, paints frames live.
- ❍ [?] Optional: matplotlib-animation renderer that outputs a GIF or MP4, embeddable in the Streamlit app.

7 Front end

The current Streamlit app (`app.py`) is a good v1 dashboard. We will extend it rather than replace it.

Near-term

- ❍ [E] Embed an animated replay of one battle on a map.
- ❍ [E] Map picker – choose which stock map to fight on.
- ❍ [E] Per-unit cover and terrain overrides in the sidebar.

Longer-term

- ❍ [?] Army builder UI: drag-and-drop datasheets, points totalled live.
- ❍ [?] Calibration browser: view the most recent sweep, drill into outlier matchups.

8 Datasheet ingestion

Importing real datasheets at scale is Jake's slice of the project. The shape of the data model needs to be agreed before he starts, otherwise we will end up rewriting `units.py` twice.

To do

- ❍ [J] Pick a source format (Wahapedia CSV / JSON dump, BattleScribe XML, hand-rolled YAML ...).
- ❍ [J] Define a target schema that maps cleanly onto `UnitProfile` once strength / toughness / move / weapon_skill / leadership are added.
- ❍ [J] Build a loader that yields `UnitProfile` instances, with a test that round-trips against a hand-coded expected unit (e.g. Intercessor squad).
- ❍ [J] Decide how multi-weapon units are flattened (single average weapon profile vs. list of weapons resolved separately).

9 Combat math reference card

9.1 Hit / wound / save chain

For one attack with weapon profile $(P_{\text{hit}}, S, \text{AP}, d)$ against target (T, sv) and cover bonus $c \in \{0, 1\}$:

$$\mathbb{E}[\text{damage}] \;=\; P_{\text{hit}} \cdot P_w(S, T) \cdot \big(1 - P_{\text{save}}(\text{sv} - \text{AP} - c)\big) \cdot d.$$

9.2 Lanchester score

$$\text{Score(unit)} \;=\; (H \cdot D \cdot P_{\text{hit}})^2, \qquad \text{Score(army)} \;=\; \sum_i e_i^2.$$

9.3 Charge probability

$$P_{\text{charge}}(d) \;=\; \frac{1}{36} \sum_{a=1}^{6} \sum_{b=1}^{6} \mathbf{1}[a+b \geq d]\,.$$

10 Collaboration workflow

10.1 Ownership at a glance

Area	Lead	Status
Simulation core (phases, math)	[E] Eddie	in progress
Datasheet ingestion + catalogue	[J] Jake	not started
Terrain & maps	[?] TBD	not started
Visualisation (dual-mode)	[?] TBD	not started
Streamlit front end	[E] Eddie	v1 done

10.2 Picking the project back up

1. `git pull`.
2. Open this PDF and scan the unchecked items in your area.
3. Pick the smallest task that unblocks something else.
4. Commit with a clear message; tick the box in this file in the same commit.

10.3 When something is safe to merge to main

- Mirror-match calibration (Space Marine vs Space Marine) still sits at $50\% \pm 2\%$ over 1000 battles.
- `python run.py` still runs end-to-end.
- The Streamlit app still launches and the preset battles still produce charts.

11 Glossary

Term	Meaning
AP	Armour penetration; negative integer degrading target save.
BS / WS	Ballistic / Weapon Skill; roll needed to hit.
Charge phase	Phase where units roll $2d6$ to close into base contact.
Cover	Terrain bonus to saving throws or hit penalty.
CP	Command Point; resource for stratagems and rerolls.
Lanchester score	$(H \cdot D \cdot P)^2$; theoretical effectiveness of a unit.
LoS	Line of sight; whether the attacker can "see" the target through terrain.
Stochastic mode	Simulator with real dice rolls.
Deterministic mode	Simulator using expected values, no rolls.